

Лабораторная работа №1

Цель работы: Изучить рекомендации по формированию паролей и их взлом с помощью атак подбором, по словарю и с применением правил.

Теоретическая часть

Формирование пароля (закрытого ключа, если он задается вручную) является важным аспектом обеспечения безопасности и оказывает существенное влияние на стойкость к взлому. При легкомысленном отношении к формированию закрытого ключа криптостойкость алгоритма шифрования отходит на второй план, т.к. становится возможным подобрать ключ (пароль), не вдаваясь в особенности шифрования.

Существует несколько простых типов атак, так или иначе связанных с перебором возможных паролей, которые могут быть применены для раскрытия пароля (закрытого ключа), если есть возможность выполнять такой перебор:

1. полный перебор (еще называется методом грубой силы – brute force). Этот метод позволяет подобрать любой пароль, однако время, необходимое для этого, может оказаться колоссальным. Действительно, если предположить, что известны допустимые символы пароля (алфавит), например, большие и маленькие латинские буквы и цифры (всего 62 символа), а длина пароля n , то количество возможных вариантов пароля $N_p = 62^n$ (при $n = 10$, $N_p = 8392993658\ 68340224$ (18 разрядов)). Если компьютер способен подбирать 1000000 паролей в секунду, то при $N_p = 8392993658\ 68340224$ для перебора всех возможных паролей понадобится около 26614 лет, что неприемлемо. Из приведенных рассуждений видно, что действенной защитой от этого вида атак является выбор достаточно длинных паролей (например, 10 и более символов) и расширение алфавита пароля (например, при добавлении больших и маленьких русских букв получаем $N_p = 128^n = 2^{7n}$, что в случае $n = 10$ дает $N_p = 2^{70} = 1180591620\ 7174113034\ 24$ (22 разряда)). Следует обратить внимание, что в вышеприведенных рассуждениях, предполагается, что при взломе известна длина пароля. Однако, если известна только максимально возможная длина пароля $n_{\max}$, то время расчета увеличивается. Например, в последнем случае, когда алфавит содержит 128 символов, а $n_{\max} = 10$, то в худшем случае потребуется перебрать

$$N_p = \sum_{n=1}^{n_{\max}} 128^n = 1189887617\ 7309342270\ 72 \text{ (всего 22 разряда) различных паролей.}$$

Следует отметить, что с учетом роста вычислительных мощностей современного аппаратного обеспечения и применением многопоточных вычислений (как на уровне CPU, так и благодаря использованию GPGPU) количество перебираемых в секунду паролей может быть существенно выше, чем 1000000. Это, в свою очередь, потребует увеличения длины пароля для сохранения устойчивости к атаке полным перебором;

2. атака по словарю. Метод предполагает, что пароль является обычным словом, т.к. его легче запомнить. Суть атаки заключается в подстановке слов из некоторого словаря, которые могут быть паролем (с учетом максимально возможной длины пароля, если она известна). Хорошей защитой от этой атаки является выбор пароля, который не является словом;
3. атака по слогам. Этот метод является разновидностью предыдущего и предполагает подбор пароля из слогов, из которых строятся слова на языке формирования пароля. Временные затраты для этого метода выше, чем для атаки по словарю. Защитой от такой

атаки является отказ от использования слов (даже несколько видоизмененных) в качестве пароля;

4. атака на основе правил. Является самой мощной из атак, направленных на раскрытие пароля. Суть атаки является в комбинации полного перебора с некоторыми эвристическими знаниями о том, как формировался пароль (атака возможно ТОЛЬКО в случае наличия таких знаний). Например, если известно, что в качестве пароля используется слово в комбинации с цифрами, то перебор можно выполнять с использованием такого шаблона. Разновидностью атаки такого вида является также приоритетность в переборе паролей: сначала могут быть рассмотрены часто используемые пароли, сочетания символов, имеющих определенное значение для человека, формировавшего пароль (день рождения, имя любимого родственника, марка любимого автомобиля, кличка собаки и т.п. Такая атака еще называется атакой на основе дней рождения).

Рекомендации по формированию пароля:

1. не задавать общеизвестные пароли типа 111, 123, qwerty и т.п., т.к. их будут перебирать в первую очередь;
2. не задавать пароли, которые имеют непосредственное отношение к формирующему их человеку. Например, дату рождения (свою или родственника), имя родственника или домашнего животного и т.п. Несоблюдение этой рекомендации может привести к существенному сокращению времени подбора паролей, если использовать атаку на основе правил, и будут накоплены данные о личной жизни задавшего пароль;
3. не использовать в качестве пароля слова, т.к. это позволяет использовать атаку по словарю или атаку по слогам;
4. не использовать один и тот же шаблон при формировании паролей (например, логин пользователя и какой-нибудь порядковый номер и т.п.), т.к. это позволяет использовать атаку на основе правил;
5. не использовать слишком коротких паролей. Например, пароль, состоящий только из больших и маленьких латинских букв и цифр (62 символа) и содержащий не более 5 символов, при атаке перебором дает $62^5 = 931151402$ возможных вариантов, что при возможности подбора 1000000 паролей в секунду потребует в худшем случае только около 15.5 минут на взлом. Хороший результат также даст расширение алфавита пароля;
6. не использовать всегда пароли одной и той же длины, особенно если это может стать известным злоумышленнику, т.к. это несколько сокращает время подбора;
7. несмотря на все вышеперечисленные требования, **пароль должен быть пригоден для запоминания**, иначе его можно забыть.

Для борьбы с взломом паролей могут быть использованы различные методы: искусственное постепенное увеличение времени ответа об удачности авторизации при превышении неудачных вводов пароля некоторого заданного значения, блокировка логина, для которого выполняется большое количество неудачных попыток подключения и т.п.

Задание

Разработать программное обеспечение, с помощью которого можно имитировать процесс взлома пароля с применением атаки перебором. **Задание выполняется группой из 2-ух студентов (один студент выступает в роли системного администратора (в дальнейшем будем называть его «Администратор»), задающего логины и пароли пользователям для удаленной работы с некоторым ПО, а второй – в роли взломщика (в**

дальнейшем будем называть его «Хакером»), который, зная логин, пытается раскрыть пароль).

Таким образом, должно быть разработано 2 приложения: серверное (разрабатывается Администратором) и клиентское (разрабатывается Хакером). Приложения могут быть как консольными, так и имеющими GUI. Взаимодействие между приложениями реализовать с помощью именованных каналов (с учетом ВОЗМОЖНОСТИ взаимодействия по локальной сети).

Серверное приложение должно:

1. при запуске запрашивать имя текстового файла, в котором заданы логины и пароли (5 пар). Файл имеет следующую структуру: 1-ая строка – максимально возможные длины логина и через пробел – пароля, а затем в каждой строке хранится логин и через пробел – пароль. Имя файла запрашивается с помощью функции **GetOpenFileName**;
2. считывать содержимое файла;
3. выбирать режим работы: обычный или с противодействием взлому (**только при выполнении на 100% или выше**);
4. после этого в **асинхронном режиме** создается **не менее 3** экземпляров именованного канала (имя канала выбрать самостоятельно, **причем на сервере и клиентах оно должно быть одинаковым**);
5. далее ожидается подключение клиентов;
6. при подключении очередного клиента считываются передаваемые логин и пароль (строковые значения). Если логин и пароль совпадают с заданными значениями, то сервер отправляет клиенту число 1, а иначе – 0. После этого клиент отключается сервером;
7. переход к пункту 5;
8. если все клиенты отключены, то сервер запрашивает необходимость продолжения работы с помощью функции **MessageBox**.
9. если пользователь выбирает продолжение работы, то переход к пункту 5.

Серверное приложение должно обеспечивать возможность подключения клиентов по сети.

При реализации режима защиты от взлома на основании увеличения времени ответа, следует отметить, что это увеличение следует реализовывать на основе пропуска обработки запросов от заданного логина в течение указанного для этого логина количества мс. Задержки в ответе для каждого логина НЕ должны влиять на скорость работы сервера в целом.

Клиентское приложение должно иметь возможность подключиться к серверу и получить сообщение об удачности авторизации. Клиент работает следующим образом:

1. сначала пользователем после соответствующего запроса вводится:
 - тип работы: проверка связи или взлом;
 - если проверка связи, то
 - вводятся тестовые логин и пароль;
 - а в противном случае производится выбор режима работы (**при выполнении задания на 100% и более**) – полный перебор и на основе правил:
в случае для полного перебора:
 - тип алфавита пароля (маленькие латинские буквы и апостроф – 27 символов, маленькие и большие латинские буквы и апостроф + цифры – 63 символа или маленькие и большие латинские буквы и апостроф + цифры + маленькие и большие кириллические буквы – 128 символов);
 - максимально допустимая длина пароля;
 - логин, для которого будет производиться взлом;

в случае атаки на основе правил (**при выполнении задания на 100% и более**):

в программе должны быть реализованы правила (в виде методов класса), которые применяются к списку наиболее вероятных паролей.

Таковыми правилами могут быть, к примеру:

а) замена букв пароля на латиницу/кириллицу (часто паролем является какое-то слово или словосочетание, которое вводится в другой раскладке клавиатуры, например, английское слово – кириллицей или русское слово – латиницей).

б) изменение регистра пароля

в) изменения порядка следования букв пароля

и т.д.

После этого вводится имя текстового файла (должно быть получено с помощью функции **GetOpenFileName**, формат этого скриптового файла разработать самостоятельно), содержащего наиболее вероятные пароли и тип правил, которые к ним применяются.

Для каждого типа правила (задается номером или идентификатором) должен быть указан список наиболее вероятных паролей, который задается именем файла, содержащего эти пароли.

После ввода имени файл считывается, причем списки паролей для каждого набора правил хранить в виде однонаправленного линейного списка. Список разных наборов правил также хранить с использованием этой же динамической структуры (использовать шаблонный класс).

2. после этого происходит попытка подключения к именованному каналу, созданному сервером. Если подключение не удалось из-за того, что он не создан сервером, выводится соответствующее сообщение об ошибке, после чего работа клиента завершается. Если же подключение не удалось по причине занятости сервера, то клиент входит в ожидание освобождения сервера и, дождавшись, выполняет подключение;
3. после подключения происходит обмен данными с сервером: **сначала клиент должен отослать серверу логин и пароль и получить от сервера подтверждение авторизации**. Если клиент работает в режиме взлома, то пункт 3 (или 2, 3) должны повторяться до тех пор, пока не будет получен положительный ответ о подключении. При выполнении взлома НЕ ДОЛЖНА применяться рекурсия;
4. если клиент работает в режиме взлома, то на экран выводится логин, найденный в процессе взлома пароля, максимально возможная длина пароля и тип используемого алфавита, а также время поиска пароля, в противном случае – сообщение об удачности или неудачности подключения;
5. после этого клиент отключается от именованного канала, и его работа завершается.

Следует отметить, что:

1. хакеру известны ВСЕ логины;
2. хакеру ИЗВЕСТЕН пароль к одному из логинов (для тестирования соединения с сервером), а пароли к остальным 4-ем логинам НЕИЗВЕСТНЫ;
3. хакеру известна максимально возможная длина, а также алфавит паролей;
4. хакеру неизвестна реальная длина паролей.
5. из 4-ех неизвестных паролей (**задаются для всех 3-ех возможных алфавитов и для 3-ех различных максимально возможных длин – определяются из расчета характеристик аппаратного обеспечения, но не менее 3-ех символов**):
 - 1-ый нарушает 1-ую рекомендацию;
 - 2-ой – 2-ую;
 - 3-ий – 5-ую;
 - 4-ый – не нарушает никаких рекомендаций.

Хакеру неизвестно, какой пароль нарушает какую рекомендацию. **Все пароли должны удовлетворять рекомендации 7;**

6. при составлении списка наиболее вероятных паролей, Хакер должен использовать эвристические предположения о том, какие пароли мог задать Администратор.

1. Для получения 75% от числа баллов, отводимых на лабораторную работу, необходимо в клиенте реализовать поиск пароля полным перебором, а на сервере не требуется реализовывать специальных действий против возможного взлома.
2. Для получения 100% от числа баллов, отводимых на лабораторную работу, необходимо в клиенте реализовать поиск пароля полным перебором и на основе правил с применением специального списка наиболее вероятных паролей, а на сервере требуется реализовывать специальные действия против возможного взлома (2 режима работы: с противодействием взлому и без противодействия).
3. *Для получения 140% от числа баллов, отводимых на лабораторную работу, необходимо в клиенте реализовать поиск пароля полным перебором и на основе правил с применением специального списка наиболее вероятных паролей, а на сервере требуется реализовывать специальные действия против возможного взлома (2 режима работы: с противодействием взлому и без противодействия). Клиент и сервер должны быть многопоточными.

При удачном взломе не менее 60% паролей (всего должно быть задано $4 \times 3 \times 3 = 36$ неизвестных паролей, 60% – 22 пароля, если используются 2 режима взлома и противодействия ему, то количество паролей умножается на 4), Хакер получает за лабораторную работу +15%, в противном случае – Администратор получает за лабораторную работу +15%.

В результате выполнения лабораторной работы ЛЮБОГО уровня сложности должен быть сделан и сдан отчет (один на 2-ух студентов), содержащий следующую информацию:

1. титульный лист;
2. перечень логинов и паролей с указанием тестового логина;
3. время раскрытия пароля к выбранному логину (указать выбранные для взлома пароль и логин; **ПОПЫТКА ВЗЛОМА ДОЛЖНА БЫТЬ ПРОВЕДЕНА ДЛЯ ВСЕХ НЕИЗВЕСТНЫХ ПАРОЛЕЙ**) с указанием типа нарушенной рекомендации (в виде таблицы);
4. количество успешно взломанных паролей, характеристики этих паролей и режимы работы клиента и сервера (в виде таблицы);
5. графики зависимости времени взлома пароля (для каждого из 4-ех паролей) в зависимости от алфавита, максимально возможной длины и с учетом, и без учета применения списка наиболее вероятных паролей и противодействия взлому (указать способ противодействия) на стороне сервера (**для уровня сложности 100% и выше**);
6. аппаратно-программные характеристики используемых компьютеров: названия и характеристики центрального процессора (частота процессора, шины, количество активных физических и логических ядер); чипсет материнской платы; тип, частота и количество оперативной памяти; характеристики локальной сети; название ОС; количество порождаемых потоков в клиентской программе;
7. выводы, содержащие мнение авторов об удачности или неудачности взлома и противодействия этому процессу (с аргументацией).

При написании программы использовать ООП. Все данные в метод передаются через его аргументы или берутся из полей класса.

Вся выводимая информация должна сопровождаться пояснениями.

Краткие сведения про WinAPI функции для работы с именованными каналами

Рассмотрим некоторые специализированные WinAPI функции, применяемые для создания именованных каналов и работы с ними, прототипы которых описаны в файле заголовков **winbase.h**, который включен в файл заголовков **windows.h**.

Функция **CreateNamedPipe** (для неименованных каналов используется функция **CreatePipe**) предназначена для создания **сервером** экземпляра именованного канала и получения его дескриптора для последующих операций (клиент получает дескриптор именованного канала с помощью функции **CreateFile**, указывая такое же имя, как и при создании канала на сервере). Серверный процесс использует эту функцию либо для создания 1-го экземпляра определенного именованного канала и настройке его характеристик, либо для создания нового экземпляра существующего именованного канала. Для удачного создания пользователь должен иметь привилегию **FILE_CREATE_PIPE_INSTANCE** по отношению к создаваемого объекту. Эта функция не может быть использована в Windows 9x (в этих ОС можно использовать именованные каналы только со стороны клиента). Ее прототип описан следующим образом:

```
HANDLE CreateNamedPipe(  
    LPCTSTR lpName,  
    DWORD dwOpenMode,  
    DWORD dwPipeMode,  
    DWORD nMaxInstances,  
    DWORD nOutBufferSize,  
    DWORD nInBufferSize,  
    DWORD nDefaultTimeOut,  
    LPSECURITY_ATTRIBUTES lpSecurityAttributes  
);
```

где **lpName** – указатель на строку, оканчивающуюся символом с кодом 0, содержащую имя создаваемого экземпляра канала, которое его однозначно идентифицирует. Имя должно быть следующего вида:

\\.\pipe\имя_канала

причем имя канала не может содержать символ «\» и превышать 256 символов (следует отметить тот факт, что при задании строковой константы на языке C++ каждый символ «\» должен быть ПРОДУБЛИРОВАН).

Аргумент **dwOpenMode** (2-ой) с помощью числовой константы задает режим доступа при создании канала. В таблице 1 приведены константы 3-ех основных режимов для именованных каналов (необходимо обязательно выбрать основной режим), а в таблице 2 – дополнительные режимы.

Таблица 1. Основные режимы доступа
именованных каналов

Режим (название константы + числовое значение)	Описание
PIPE_ACCESS_DUPLEX 0x00000003	Создается двунаправленный именованный канал, для которого допустимо как чтение, так и запись для клиента и сервера (аналогично

	режиму работы с файлом GENERIC_READ GENERIC_WRITE). Клиент при использовании функции CreateFile может задавать любую комбинацию прав GENERIC_READ и GENERIC_WRITE
PIPE_ACCESS_INBOUND 0x00000001	Клиент может только записывать в канал (должен использовать режим GENERIC_WRITE), а сервер – только считывать данные из канала (GENERIC_READ).
PIPE_ACCESS_OUTBOUND 0x00000002	Сервер может только записывать в канал (GENERIC_WRITE), а клиент – только считывать данные из канала (должен использовать режим GENERIC_READ).

**Таблица 2. Дополнительные режимы
именованных каналов**

Режим (название константы + числовое значение)	Описание
FILE_FLAG_FIRST_PIPE_INSTANCE 0x00080000	Указывается при создании 1-го экземпляра канала (при попытке создания 2-го экземпляра с указанием этого флага произойдет ошибка). Этот флаг нельзя указывать, если ОС Windows 2000 SP1 или ниже
FILE_FLAG_WRITE_THROUGH 0x80000000	Режим «прямой» записи данных, минуя кеширование
FILE_FLAG_OVERLAPPED 0x40000000	Асинхронный режим работы

В этом аргументе также можно задавать флаги безопасности (**WRITE_DAC**, **WRITE_OWNER** и **ACCESS_SYSTEM_SECURITY**).

Следует обратить внимание на тот факт, что при необходимости общения с несколькими клиентами, подключение которых могут быть не синхронны, рекомендуется создавать экземпляры канала (по числу одновременно подключенных клиентов) именно в асинхронном режиме работы.

Аргумент **dwPipeMode** (3-ий) задает режим работы именovanного канала (интерпретация данных при записи и при чтении, режим ожидания). Режимы работы представлены в таблице 3.

**Таблица 3. Режим работы именovanных
каналов**

Режим (название константы + числовое значение)	Описание
PIPE_TYPE_BYTE	Данные записываются в канал, как поток

0x00000000	байт. Несовместим с режимом PIPE_READMODE_MESSAGE .
PIPE_TYPE_MESSAGE 0x00000004	Данные записываются в канал, как поток сообщений. Совместим с режимами PIPE_READMODE_BYTE и PIPE_READMODE_MESSAGE
PIPE_READMODE_BYTE 0x00000000	Данные считываются из канала, как поток байт. Совместим с режимами PIPE_TYPE_BYTE и PIPE_TYPE_MESSAGE .
PIPE_READMODE_MESSAGE 0x00000002	Данные считываются из канала, как поток сообщений. Совместим с режимами PIPE_TYPE_BYTE и PIPE_TYPE_MESSAGE .
PIPE_WAIT 0x00000000	Разрешается режим блокировки. Когда дескриптор канала указывается в функциях ReadFile , WriteFile или ConnectNamedPipe , то функция не завершается пока, данные не будут прочитаны или записаны (все данные) или клиент не подключится к каналу.
PIPE_NOWAIT 0x00000001	Режим блокировки отключен. Не рекомендуется использовать с асинхронным вводом-выводом.

Все экземпляры именованного канала должны иметь один и тот же режим работы, иначе функция завершится неудачно, а функция **GetLastError** вернет значение **ERROR_ACCESS_DENIED**.

Следует обратить внимание на тот факт, что на стороне клиента вывод в канал начинается в байтном режиме, даже если на стороне сервера выбран режим сообщений. Чтобы избежать проблем с приемом данных следует установить режим сообщений также и для клиента.

Аргумент **nMaxInstances** (4-ый) задает максимальное количество экземпляров канала, которое может быть роздано. Может лежать на отрезке от 1 до **PIPE_UNLIMITED_INSTANCES** (значение 255). Если задать значение больше, чем **PIPE_UNLIMITED_INSTANCES**, то функция завершится неудачно, а функция **GetLastError** вернет значение **ERROR_INVALID_PARAMETER**.

Аргументы **nOutBufferSize** и **nInBufferSize** (5-ый и 6-ой) задают размеры выходного и входного буферов, соответственно. Эти размеры носят рекомендательный характер и могут быть изменены ОС автоматически. Однако эти размеры все же следует выбирать соответственно решаемой задаче (они не должны быть слишком большими, чтобы исключить бесполезную трату памяти, и слишком маленькими, чтобы не снижать эффективность кеширования). При недостаточном размере буфера ОС попытается его автоматически увеличить, но это приведет к временной блокировке операции.

Аргумент **nDefaultTimeOut** (7-ой) задает значение по умолчанию для времени ожидания при использовании в качестве 2-го аргумента функции **WaitNamedPipe** значения **NMPWAIT_USE_DEFAULT_WAIT**. При создании всех экземпляров одного и того же канала должно использоваться одинаковое значение.

Аргумент **lpSecurityAttributes** (8-ой) задает указатель на структуру **SECURITY_ATTRIBUTES**, содержащую дескриптор безопасности. В качестве этого аргумента можно передать **NULL**, что не позволит наследовать дескриптор дочерним процессам.

В случае удачного выполнения функция возвращает дескриптор именованного канала, а в противном случае – значение **INVALID_HANDLE_VALUE**. Когда последний дескриптор экземпляра именованного канала будет закрыт (функцией **CloseHandle**), то этот экземпляр будет автоматически удален.

Функция **ConnectNamedPipe** позволяет серверу ожидать, пока клиент подключится с помощью функций **CreateFile** или **CallNamedPipe** к именованному каналу. Ее прототип описан следующим образом:

```
BOOL ConnectNamedPipe(  
    HANDLE hNamedPipe,  
    LPOVERLAPPED lpOverlapped  
);
```

где **hNamedPipe** – дескриптор канала, полученный с помощью функции **CreateNamedPipe**, а **lpOverlapped** – указатель на структуру **OVERLAPPED**, которая используется при асинхронном режиме работы с каналом (в синхронном режиме в качестве 2-го аргумента можно передать **NULL**). Структура **OVERLAPPED** должна содержать дескриптор события с РУЧНЫМ сбросом.

Следует отметить, что при необходимости соединения с несколькими клиентами рекомендуется использовать именно асинхронный режим работы с каналом.

В случае удачного завершения функция возвращает не 0, а в противном случае – 0. Если клиент подсоединился после вызова **CreateNamedPipe**, но до вызова **ConnectNamedPipe**, то функция вернет 0, а **GetLastError** – **ERROR_PIPE_CONNECTED**. Однако в описанном случае на самом деле ошибка отсутствует. Поведение функции зависит от того, включена для канала блокировка или нет. Если блокировка отключена, то функция завершается немедленно и возвращает 0, а **GetLastError** – расширенный код ошибки: **ERROR_PIPE_LISTENING** – клиент не подсоединился, **ERROR_PIPE_CONNECTED** – клиент подсоединился и **ERROR_NO_DATA** – если канал был подключен к другому клиенту, который закрыл дескриптор на него, но сервер не вызвал функцию **DisconnectNamedPipe**, осуществляющую отсоединение серверного конца канала, прототип которой описан следующим образом:

```
BOOL DisconnectNamedPipe( HANDLE hNamedPipe );
```

где **hNamedPipe** – дескриптор канала, полученный с помощью функции **CreateNamedPipe**. В случае удачного завершения функция возвращает не 0, а в противном случае – 0. Перед вызовом этой функции серверу рекомендуется вызвать функцию **FlushFileBuffers**.

Функция **WaitNamedPipe**, как правило, применяется на стороне клиента для ожидания в течение указанного промежутка времени доступности канала для подключения (т. е. факта вызова сервером функции **ConnectNamedPipe**). Ее прототип описан следующим образом:

```
BOOL WaitNamedPipe(  
    LPCTSTR lpNamedPipeName,  
    DWORD nTimeOut  
);
```

где **lpNamedPipeName** – строка, заканчивающаяся символом с кодом 0, которая содержит имя канала (см. описание функции **CallNamedPipe**), а **nTimeout** – время ожидания доступности канала в миллисекундах (см. описание функции **CallNamedPipe**).

В случае удачного завершения функция возвращает не 0, а иначе – 0. Если экземпляра указанного именованного канала не существует, то функция заканчивается немедленно независимо от указанного времени ожидания.

Функция **SetNamedPipeHandleState** позволяет изменить режимы чтения и блокировки для указанного канала. Если функция вызвана клиентом и клиентский и серверный процессы находятся на разных компьютерах, то функция также позволяет управлять кешированием канала. Ее прототип описан следующим образом:

```
BOOL SetNamedPipeHandleState(  
    HANDLE hNamedPipe,  
    LPDWORD lpMode,  
    LPDWORD lpMaxCollectionCount,  
    LPDWORD lpCollectDataTimeout  
);
```

где **hNamedPipe** – дескриптор канала, открытый в режиме **GENERIC_WRITE**.

Аргумент **lpMode** (2-ой) задает указатель на ячейку памяти, в которую будет записан новый режим канала. Допустимые значения: **PIPE_READMODE_BYTE**, **PIPE_READMODE_MESSAGE**, **PIPE_WAIT** и **PIPE_NOWAIT**, которые описаны в таблице 3. Может быть **NULL**, если режим не изменяется.

Аргумент **lpMaxCollectionCount** (3-ий) задает указатель на ячейку памяти, в которую будет записано новое максимальное количество байт, которое может быть накоплено клиентом перед отправкой на сервер. Этот аргумент должен быть **NULL**, если функция вызывается сервером или клиент и сервер находятся на одном компьютере. Это значение игнорируется, если дескриптор канала был получен с указанием флага **FILE_FLAG_WRITE_THROUGH** в функции **CreateFile**. Может быть **NULL**, если значение не изменяется.

Аргумент **lpCollectDataTimeout** (4-ый) задает указатель на ячейку памяти, в которую будет записано новое значение максимального времени, за которое может пройти отправка данных по сети. Этот аргумент должен быть **NULL**, если функция вызывается сервером или клиент и сервер находятся на одном компьютере. Это значение игнорируется, если дескриптор канала был получен с указанием флага **FILE_FLAG_WRITE_THROUGH** в функции **CreateFile**. Может быть **NULL**, если значение не изменяется.

В случае удачного завершения функция возвращает не 0, а в противном случае – 0.

Таким образом, простейший алгоритм работы с именованными каналами следующий:
На стороне сервера:

1. создание нужного количества экземпляров заданного именованного канала с помощью функции **CreateNamedPipe**;
2. ожидание соединения с клиентами с помощью функции **ConnectNamedPipe**;
3. обмен данными с клиентами;
4. отсоединение клиента (при необходимости) при помощи функции **DisconnectNamedPipe**;
5. в конце работы – освобождение ресурсов из-под дескрипторов при помощи функции **CloseHandle**.

На стороне клиента:

1. получение дескриптора для УЖЕ СОЗДАННОГО экземпляра именованного канала с помощью функции **CreateFile** (либо выполнение всех действий: открытие, ожидание соединения, чтение-запись, освобождение ресурсов функцией **CallNamedPipe**);

2. ожидание соединения с сервером при помощи функции **WaitNamedPipe** (при необходимости), если соединение с сервером не получилось при вызове функции **CreateFile** по причине неготовности сервера.
3. изменение режима работы канала (при необходимости) при помощи функции **SetNamedPipeHandleState**;
4. обмен данными с сервером;
5. в конце работы – освобождение ресурсов из-под дескрипторов при помощи функции **CloseHandle**

Следует обратить внимание на тот факт, что использовать синхронный режим работы именованных каналов при создании приложений, выполняющих обмен данными с помощью этого механизма, целесообразно в том случае, когда сервер ждет подключения одного клиента и, дождавшись, выполняет обмен данными с ним, после чего клиент отключается и т. д.

Однако если серверу нужно работать с несколькими клиентами одновременно, то целесообразнее применять асинхронный режим работы с именованными каналами (в серверном приложении). В этом случае появляется возможность в асинхронном режиме ждать подключения одних клиентов, а в это же время работать с другими, уже подключенными, клиентами.

При использовании асинхронного режима перед началом чтения / записи поля **Offset**, **OffsetHigh**, **Internal**, **InternalHigh** нужно обнулить, но после каждой операции чтения / записи ИЗМЕНЯТЬ ПОЛЯ **Offset** и **OffsetHigh** ВРУЧНУЮ НЕ НУЖНО. Система автоматически изменяет поля **Internal** и **InternalHigh**. Состояние событий функции для работы с каналами меняют АВТОМАТИЧЕСКИ.

Рассмотрим пример использования именованных каналов (см. проект к лабораторной работе).

Пример №1. Написать два приложения: клиентское, которое определяет значения чисел кратных заданному значению в заданном диапазоне (эти данные вводятся пользователем) и с применением именованных каналов отправляет серверному приложению границы диапазона, делитель и полученные числовые значения. Серверное приложение ожидает соединения с клиентами и, соединившись, получает через именованные каналы передаваемые клиентом данные, которые записывает в файл (имя файла задается пользователем).

При решении поставленной задачи серверное приложение будет использовать асинхронный режим работы с именованными каналами, причем именованные каналы будут работать в байтном режиме. Именованные каналы будут создаваться однонаправленными (клиент записывает данные, сервер их считывает). Получаемые от каждого клиента числовые значения серверное приложение будет записывать в однонаправленный линейный список, для работы с которым будет использован шаблонный класс **TList**.

Серверное приложение.

Таким образом, проект серверного приложения будет содержать следующие файлы: **list.h** – содержит описание шаблонного класса для работы с однонаправленным линейным списком и класс-итератор; **PipeServer.h** – содержит описание шаблонного класса, реализующего базовые возможности для работы с именованным каналом в асинхронном режиме на стороне сервера; **PerPipeStruct.h** – содержит описание шаблонного класса, необходимого для интерпретации полученных данных для каждого экземпляра именованного канала; **test.cpp** – содержит функцию **main**.

Клиентское приложение.

Проект клиентского приложения будет содержать следующие файлы: **PipeClient.h** – содержит описание шаблонного класса, реализующего базовые возможности для работы с

именованным каналом в синхронном режиме на стороне клиента; **test.cpp** – содержит вспомогательную функцию и функцию **main**.

Если клиент и сервер запускаются на одном и том же компьютере и при подключении клиента указать имя компьютера, отличное от «.», то при неподключенной сети клиент выдаст ошибку с кодом 1231 (сеть недоступна), если не установлен сетевой протокол «NWLink IPX/SPX/NetBIOS транспортный протокол» (Windows XP). Если же вышеупомянутый сетевой протокол установлен, то в этой ситуации клиент выдаст ошибку с кодом 53 (неправильный сетевой путь).